The Model Knows the Textbook but Not Your Class: Curriculum Knowledge Graph Constraints for Slide Generation in Two Vocational-Education Courses

Suxing Liu[a,*]

[a]*School of Digital Media Technology, Jiangxi Arts & Ceramics Technology Institute, 538 Xinchang West Road, Zhushan District, Jingdezhen, 333001, Jiangxi, China*

Abstract

Evaluating generated teaching material raises a question benchmarks do not: does the artefact satisfy its course specification? We make it computable. Course-level knowledge—what a session must cover, what this cohort mislearns, what must not appear yet—is compiled from a curriculum graph into an executable contract that doubles as the acceptance criterion, so requirement and check cannot drift.

We study the queue module of a vocational *Python Data Structures* course across three backbones, three ablations and a second course in another language. The change is reliability, not average quality: misconception handling reaches 1.000 with zero dispersion against 0.556 ± 0.527 for a text-matched control, surviving session clustering. Coverage rises from 0.61–0.70 to 0.98 where the backbone follows structural instructions; a length-matched placebo excludes prompt length, though a three-cluster interval includes zero. Ablation prices each block: removing the misconception slot takes it to 0.167, the forbidden list buys 0.083 of out-of-graph control.

The second course keeps the sign of that contrast but not its reliability, and its coverage gain over that control shrinks. Re-running the first course under that backbone alone removes the advantage, placing the condition on the model, not the syllabus: a contract can require a slot; only a compliant backbone fills it every time. Ordering is where the representation stops—expressible in the graph, not withholdable by the interface—so

*Corresponding author.

Email address: bentondoucet@gmail.com (Suxing Liu)

its null ablation is a boundary, not a failure. Seven measurement defects favouring the constrained condition are documented; a blind review matches the automatic ranking (ICC(2,3) = 0.92).

Keywords: curriculum knowledge graph, constrained generation, slide automation, misconception modelling, educational knowledge engineering

---

1. Introduction

1.1. A concrete failure

Asked for the slides of a session on queues, a general-purpose large language model (LLM) returns a well-organised deck with accurate terminology and runnable code. A teacher skimming it nods; a teacher reading it closely frowns. Three failures recur.

- The objective says students must *implement* enqueue and dequeue on a sequential queue using a Python list. The deck never shows **how to write it**: it covers the definition, the false-overflow problem and the circular queue as the remedy—everything except the step the syllabus requires students to perform.
- The session carries a misconception that more than 60% of this cohort reproduces in the final exam: "a Python `list` is a linked list underneath." The model does not mention it, because it is not a textbook topic but a property of *these* students.
- The model moves the circular queue's modular arithmetic onto the sequential-queue slide. Framed as "broadening the view," it inverts the prerequisite order: students meet `% maxsize` before they know what `rear` is.

None of the three is a generation-capability problem. The model can write this content; it does not know what *this course* requires, where *this cohort* errs, and how *this institution* sequences its sessions.

1.2. Thesis

The bottleneck has shifted from *knowledge acquisition* to the **representation and application of course-level knowledge**. A general model holds disciplinary knowledge (what a queue is, why FIFO matters); slide production needs *situated* knowledge, of three kinds. It is *cohort-specific*—this year's students confuse the empty and full conditions above 60%, a fact that

lives in teacher experience and assignment statistics and in no public corpus; *institution-specific*—our department merges "linked queue" and "introducing BFS" into one session, a scheduling decision that varies by institution; and *landing-point specific*—this course requires "can write the code", not "can state the concept", which the curriculum standard sets and the textbook does not. None can be obtained from a larger model or a better prompt; all must be **supplied from outside**. The question is in what form, such that the model can follow it *and* the system can afterwards check whether it did.

Two claims are on offer here, and they have different lifespans. The first is an *effect claim*: a graph-compiled contract improves generated decks along named dimensions (§5.4). The second is a *scope claim*: within a curriculum graph, only some dimensions can be compiled into requirements that are simultaneously generation constraints and acceptance criteria, and the boundary between the two sets is measurable rather than a matter of taste (§1.5). The effect claim is bounded by two courses in one department and will need replication; the scope claim is not, because it is a property of the representation rather than of the syllabus. We try to keep the two apart throughout, and where a result belongs to the first we say so.

1.3. Our answer

A **knowledge graph**, because it satisfies four requirements at once. It is *expressive* enough for heterogeneous entities (misconceptions, skills, competence levels); `prerequisites_for` is a directed edge, so a topological sort *is* a legal teaching order; one session's subgraph *compiles* mechanically into a generation contract; and that contract is *reusable as ground truth*, so what is demanded and what is checked share one source and cannot drift. The fourth is the difference from "writing the key points into the prompt": a prompt is discarded once written and compliance cannot be recovered from it afterwards, whereas a contract is a data object readable by programs both before and after generation.

1.4. Contributions

1. **A curriculum-graph schema for vocational programming courses.** We extend the node skeleton of K12-KGraph [1] by promoting `Misconception` to a first-class node, and add two edge types: `addresses_misconception`, which generates the misconception section of the contract and is scored by `misconception_coverage`; and `implements`, which today supplies only the evidence anchor the prompt

cites for a concept and is read by no headline metric (§7.6). The two are not used symmetrically, and the asymmetry is stated rather than glossed.

2. **Constraint-as-data: one object for the demand and for the check.** `UnitConstraints` drives both prompt construction and post-hoc validation: order, required/optional split, sibling-recap caps and forbidden-term lists are derived mechanically from the subgraph. No *session-specific* rule is hand-written—the seven layout rules, the JSON schema and the importance threshold are authored once, held constant across every condition, and listed in Table 1. What this buys is not tidiness, and the difference from "one schema for the prompt and one for the validator" is testable: if the two are merely co-located, a mistake introduced into the checker is invisible, whereas here the requirement and the check are one data structure read twice, so a mistake has one place to live and the ablation can price it. The observable consequence is empirical rather than aesthetic—we found **seven** measurement defects in our own pipeline, each written independently of the contract and each silently favouring the constrained condition (§7.1), and the ablation then attributes the surviving gains to named components one at a time (Table 5).
3. **A scope claim for constraint-as-data, and the method that measures its edge.** Only those dimensions of a curriculum graph that compile into requirements on the *structure* of the output—presence, absence, a corrective slot—can serve at once as generation constraint and acceptance criterion. Order is representable in the graph but, in the interface tested here, not withholdable from the model, so it falls outside the method's range. The edge is not asserted but measured: the same ablation that prices a component also identifies the dimension on which the representation runs out (§1.5, §5.4). This is the claim we expect to outlive the course, because it is a property of the representation rather than of the syllabus.
4. **A reproducible six-metric acceptance protocol**, two metrics of which are redefined for this task: prerequisite order is judged by *where a concept is named* rather than by model-reported identifiers, and coverage is matched against *graph synonyms* so that a condition copying the source wording gains no advantage.
5. **An empirical study whose headline is a second moment, with every gain attributed to a named component.** Six conditions × three

sessions × three resamplings (54 runs), three ablations (18 runs) and a length-matched placebo, in one course, with the claim-bearing conditions re-run on a second vendor's model and the two contrasts replicated on a second course in another language (§5.6). Misconception handling differs in the mean *and* in the variance (1.000 ± 0.000 against 0.556 ± 0.527), and the collapse of dispersion is the result, because a preparation workflow cannot absorb a coin flip; coverage improves by about 0.3 with the length explanation excluded by the placebo. The ablation then prices each block separately (Table 5): 0.833 of misconception coverage for the slot, 0.083 of out-of-graph control for the forbidden list, and one documented side effect, the slot's 0.056 of coverage. **Seven** measurement defects that systematically favour constraint-based methods are documented together with their fixes (§7.1). Two courses bound what can be generalised, and §7.2 says where.

6. **A blind teacher review that independently reproduces the automatic ordering.** Three teachers scored ten decks on five five-point items and an overall score, yielding ICC(2,3) = 0.92; rescaled to $[0, 1]$, their misconception ratings fall within 0.09 of the automatic metric on all four conditions, and they place the difference between conditions on the same axis: misconception handling and coverage, not factual accuracy.

1.5. The scope of the claim: what compiles into a criterion

Constraint-as-data is not a claim about curriculum knowledge in general. It is a claim about the part of a syllabus that can be compiled into a hard requirement on the *structure* of an artefact, and the edge of that part is a result of this paper rather than a caveat attached to it. Three dimensions compile. *Presence*: a required concept is named, with graph synonyms counted. *Absence*: a term belonging to a later session does not appear. *Slot*: corrective content for a named misconception occupies a page of a particular kind. Each becomes an inequality over the finished deck, and each is falsifiable without a human in the loop.

A fourth dimension is representable but not compilable in the interface we tested. Order lives in the graph as a directed `prerequisites_for` edge, and a topological sort *is* a legal teaching order, so the requirement is writeable. But the prompt must list the concepts, and listing them in topological order already carries the order; an arm that deletes the explicit ordering sentence deletes a restatement of what remains. What a graph can *say* about a syllabus and what a prompt can *withhold* are therefore different sets, and

this paper reports the difference instead of the membership we would have preferred.

The generalisation on offer is consequently narrower than "graph constraints improve generated teaching material" in one direction and wider in another. For a given professional specification, some of its dimensions yield requirements a program can both impose and verify, and some yield requirements that can only be imposed; which is which is an empirical property of the representation chosen, and it is measurable by exactly the ablation of §5.4(5). A scope statement that predicts its own nulls is therefore more useful to the next system builder than a table of wins, and it is the part of this paper that would survive a change of course, textbook or model. It is also what keeps the positive results honest: a protocol that returned a win on all four dimensions would be reporting its own bias, and seven such biases had to be removed before any result could be stated at all (§7.1).

One qualification belongs to the claim rather than to a caveat on it, and §5.6 is what forces it. The boundary is a property of the representation; its *binding force* is not. A dimension can compile into a criterion and still fail to bind on a backbone that will not execute a structural instruction—on the third backbone we tested the set of compilable dimensions is unchanged while the reliability of one of them collapses. The scope claim is therefore stated for a backbone that follows structural requirements: *given such a backbone*, which dimensions compile into criteria is fixed by the representation and measurable by the ablation above. Naming the executor is not a weakening. A boundary that silently assumed a compliant model would be the same error as a table of wins—a property of the experiment presented as a property of the method.

1.6. Why this belongs in Knowledge-Based Systems

The object of this paper is not "a better slide generator" but **how knowledge must be represented in order to be actionable by a machine**: what structure suffices to constrain generation, how it is compiled, how it is verified, and along which dimensions a knowledge base is genuinely irreplaceable. Three recent papers in this journal set the context. Dai et al. [2] show that the *presentation format* of a graph changes what a model comprehends; Huang et al. [3] show that triples and passages are not interchangeable for long-tail facts, and that combining them reduces hallucination; Ma et al. [4] place education among the journal's active applications. We continue this line but move the question from *whether knowledge improves output* to

whether the same knowledge can serve as the acceptance criterion. We also report where it does not help: a graph is not superior along every dimension, and identifying *which* knowledge must be externalised is more useful than a blanket claim that augmentation works.

The same move is being made from the other side of the field. Roschelle et al. [5] argue that responsible AI in education needs the learning sciences rather than benchmark scores, which is a claim about what the criterion should be *about*; this paper is an attempt to say what the criterion should be *computed from*, and to show what happens to a measurement once the two are the same object.

2. Related work

2.1. Automatic slide generation

Document-to-slide work measures *fidelity to the input*. DOC2PPT [6] formalised the task—summarisation, figure–text retrieval and layout arrangement at once—and released 5,873 paired examples with four metrics. Mondal et al. [7] added end-user specifications, so that one paper yields different decks for different audiences or durations, and cut preparation time from about an hour to under thirty minutes, while stating that they do not address layout design. Instruction-to-slide work measures *fidelity to editability*: AutoPresent [8] released SlidesBench (7k training, 585 test examples) and found programmatic generation superior to image generation because the artefacts stay re-editable; PPTAgent [9] analyses reference decks and then emits editing actions, with PPTEval scoring content, design and coherence. Our rendering choices follow this strand.

Evaluation has moved in two steps. PPTC [10] asks whether a model can modify a deck correctly, comparing file state rather than API calls, and finds GPT-4 at 75.1% single-turn but 6% session accuracy. More recently, evaluation has become itemised: PresentBench [11] pairs 238 instances with an average of 54.1 hand-written binary checklist items each, DeckBench [12] applies the same logic to multi-agent academic decks, PPTBench [13] isolates layout and design understanding, SlideSpawn [14] covers the research-paper pipeline, and Talk to Your Slides [15] treats editing as structured-data manipulation. This is where our evaluation belongs, one step short of our question: their rubrics are authored *per instance, by an annotator reading the source material*, whereas ours must be authored *per course, by the curriculum* and

be identical for every deck of the same session—otherwise the criterion moves with the artefact.

Our difference. Their input is a document to compress or a single instruction, and the criterion is resemblance, appeal or task completion. Our input is a course specification (what to teach, in what order, which misconception to correct, what not to introduce), and the criterion is conformance. Neither SlidesBench protocol can express "the deck omits the skill the syllabus requires students to implement," which is the most damaging class of failure in a teaching context.

2.2. Educational knowledge graphs

K12-KGraph [1] is the closest knowledge-side work: nine node types and fourteen relation types over the official People's Education Press textbooks, from which K12-Bench (23,640 items) and a graph-guided fine-tuning corpus are derived. Its most telling number is that even the strongest proprietary model reaches only 57% exact match, with Prereq and Neighbor the hardest families—evidence that structural curriculum cognition is a capability dimension independent of factual recall. SciMKG [16] builds a multimodal graph over open courseware (1,356 knowledge points, 34,630 concepts, 403,400 triples), and Ma et al. [4] apply knowledge-enhanced LLMs to course recommendation.

The centre of gravity in that literature is *building* graphs; ours is *using* them. The two are complementary, and we reuse the K12-KGraph skeleton rather than replace it. For vocational education a graph must additionally carry **competence level** (which decides mandatory versus optional) and **typical misconceptions** (which decide corrective content); K12-KGraph has an importance attribute it does not use to constrain generation, and no misconceptions at all. The retrieval side divides differently: GraphRAG [17] and the wider RAG literature [18, 19] treat a graph as a *retrieval index* that decides what the model reads. Here it is an *obligation structure* that decides what the output must contain. A wrong retrieval still permits a correct answer; a violated obligation is a defect by definition.

2.3. Curriculum specifications as the criterion

Judging a teaching artefact against the course's stated intentions is the central claim of constructive alignment, and the current statement of that claim is narrower than it is often taken to be: alignment is not the co-presence

of outcomes, activities and assessment but the *validity* of the inference from one to the next, which is why learning design and the evidence that operationalises it are treated as prerequisites rather than as decoration [20, 21]. We take the demand literally and ask what changes when the aligned object must be *computed* rather than written: if the criterion is a program, alignment stops being a property of a course design and becomes a property of each artefact.

Encoding a unit of learning as an executable specification is an old ambition: learning-design formalisms described roles, activities and sequencing precisely enough for a conforming runtime. That line never met a generative model, for an instructive reason—the specification had to be authored by a human designer, because nothing else could write content against it. LLMs change the economics: the specification becomes cheap to satisfy but not to supply. We keep the ambition and narrow the scope to the subset a deck can be graded against. What distinguishes our use of a schema is not the schema but its provenance and its second life: it is compiled per session from a course graph, and then reused as the acceptance criterion rather than only as input.

This also constrains what the contract may ask for. ICAP orders engagement from passive through active and constructive to interactive, and a synthesis of that line puts the learning-outcome advantage of the higher levels on a quantitative footing [22]; a meta-analysis of college students in the humanities and social sciences finds the same direction for replacing lecture time with students doing something themselves [23]. A deck is a passive artefact, which is why the contract carries a skill landing point and a lab rather than only a concept list—and why a deck can score 1.000 on every content metric and still be a poor lesson (§7.2).

2.4. Constrained generation

Survey accounts of natural language generation keep content planning and surface realisation apart [24], and the hallucination literature documents what happens when the two collapse: generators produce fluent text whose content the input does not support, and the error is intrinsic rather than a matter of style [25]. Knowledge-enhanced generation answers this by putting an explicit knowledge source into the pipeline, whether at training or in the prompt [26]. What unites the line is that structure enters the training objective or the decoder. In the LLM era it enters the prompt, and the form matters: Dai et al. [2] show that the same graph presented as triples,

paragraphs or neighbourhoods is comprehended with markedly different effectiveness, and Huang et al. [3] show that triples generally beat passages on long-tail facts while combining them reduces hallucination.

A stronger form acts on the decoder. Grammar-constrained decoding [27] intervenes at each step so that output cannot leave a formal language, and generalises well beyond parsing; we deliberately do not use it. A grammar can express that a slide has a title and at most four bullets, but not that this session must correct the false-overflow misconception. Those constraints are over *content*, so enforcement happens after generation or in the prompt—which is why acceptance must be a computation over the finished deck. Where a constraint is genuinely syntactic (the JSON schema, the layout enumeration, the bullet cap) we enforce it structurally and the validator reports it as a structural finding, not a content defect.

Evaluation has converged on reference-free metrics such as faithfulness [28], which ask whether every claim is supported by the retrieved context. That is the right question for "is it grounded" and silent on ours: a deck can be faithful to a passage and still omit the required skill, or reteach what the previous session covered. A parallel line replaces the metric with a judge—LLM-as-a-judge [29] and rubric-conditioned scoring [30]—now correlated well enough with human preference to serve as a primary evaluation. We use a judge only for the qualitative pass and keep the headline numbers mechanical, because a fixed criterion must yield a fixed score and a judged score cannot promise that.

We differ from both lines: we neither train, nor modify decoding, nor rerank. We compile constraints into a data object, use it to construct the prompt before generation, and use the same object to validate afterwards. The price is that we relinquish control over decoding and must bear the risk of disobedience; §5.8 reports a real violation, and §7.1 the seven traps this permissiveness exposed.

2.5. The same premise, industrialised

Learning Commons, an initiative of the Chan Zuckerberg Initiative, ships two of the components this paper argues for: a *Knowledge Graph* of academic standards, skills and learning progressions for the four core K-12 subjects across all fifty US states, and *Evaluators* that score AI-generated educational content against expert-authored rubrics [31]. More than seventy partners have integrated it, including several model providers. Two consequences follow: the question this paper asks is now an engineering need with users rather

than a hypothetical, and the architecture chosen there is the one argued for here—a graph that holds the obligations, separated from an evaluator that checks against them, so that the two cannot drift.

The difference is scope. A standards graph covers publicly codified expectations, where the mapping from standard to content is many-to-one and stable across districts. A vocational course's obligations are largely local: which skill *this* cohort must implement at the end of *this* session, which misconception *this* class holds, what the previous session already covered. None of that is in a standards document and none can be purchased as a dataset—which is why the contract carries a skill landing point, a lab, a sibling set and a misconception assignment, and why the graph must be authored by the teacher. The layers are complementary: a standards graph could populate the upper part of our schema, and the local layer is what a teacher adds.

3. Problem formulation

3.1. Curriculum knowledge graph

A curriculum knowledge graph is a typed directed heterogeneous graph $G = (V, E, \tau)$, where $\tau : V \to T_V$ with

$$T_V = \{\text{Course}, \text{Module}, \text{Unit}, \text{Concept}, \text{Skill}, \text{Lab}, \text{Exercise}, \text{Misconception}\};$$

each edge $e = (u, r, v) \in E$ carries a relation type $r \in T_E$ with

$$\begin{aligned} T_E = \{&\text{is_a},\ \text{prerequisites_for},\ \text{relates_to},\ \text{implements},\\ &\text{verified_by},\ \text{assessed_by},\ \text{addresses_misconception},\\ &\text{appears_in},\ \text{is_part_of},\ \text{leads_to}\}; \end{aligned}$$

and selected edges carry an `evidence` field recording the provenance of the relation (textbook page, curriculum-standard clause, error-rate statistic).

Constraint 1 (acyclicity). The subgraph induced by `prerequisites_for` must be a DAG. The graph layer enforces this: every cycle is reported as a `PREREQ_CYCLE` error and the loader refuses a graph that fails validation, so a circular prerequisite stops the pipeline before any generation happens. The compiler retains a defensive branch that falls back to declaration order, unreachable for a validated graph and pinned by two unit tests; a compiler should not crash mid-run, but a curriculum error must never be compiled silently into a prompt.

Table 1: Components of the session contract $C(u)$ and where each is read from in the graph.

Symbol	Meaning	Source
$S_u = (R_u, O_u)$	required / optional concept sets	split on `importance`; at or above "master" is required
$\prec_u$	teaching order (a total order on S_u)	topological order of `prerequisites_for` over G_u
M_u	misconceptions that must be corrected	`Misconception` nodes targeted by `addresses_misconception`
F_u	forbidden-term list	`forbidden_terms` on the Unit node
B_u	sibling-session concepts admissible as brief recap	concepts of other Units in the same Module, deduplicated
k_u	slide budget	`slide_budget` on the Unit node

3.2. Session contract

For a session $u \in V$ with $\tau(u) = \text{Unit}$, take the subgraph

$$G_u = \{\, v \in V : (v, \text{appears_in}, u) \in E \,\} \tag{1}$$

and compile it into a contract

$$C(u) = \langle S_u,\ \prec_u,\ M_u,\ F_u,\ B_u,\ k_u \rangle, \tag{2}$$

whose components are given in Table 1.

B_u and F_u must not be conflated: the former means "already covered, may be mentioned in one sentence", the latter "not taught in this course at all". Conflating them makes two metrics fight over one term, so the implementation enforces mutual exclusivity by assertion and §4.2 checks it on the graph.

Compilation adds no information; it selects and orders what the graph already contains. Fig. 2 shows the U11 subgraph and the contract compiled from it: every edge in panel (a) maps to a field in panel (b), and nothing in panel (b) is typed by hand. "Lossless" is meant in this narrow sense, with two recorded exceptions: the required/optional split applies a policy constant (the `importance` level at which a concept becomes mandatory), and Kahn's algorithm breaks ties by declaration order, so the compiled order is one of several admissible topological orders. Both are visible in $C(u)$ but they are policy, not data.

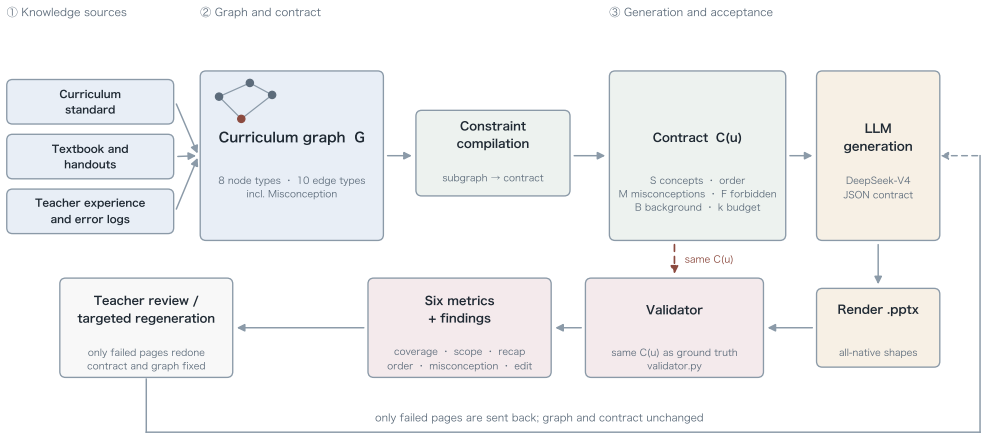


Figure 1: Framework overview. The contract $C(u)$ is read once before generation (to build the prompt) and once after (to compute the six metrics); both reads concern the same object, which is what "demand and check share one source" means in implementation terms.

3.3. Generation and acceptance

Generation is a mapping $D = f_\theta(C(u))$ where D is a sequence of slides. Acceptance is a metric vector

$$m(D, C(u), G) = \langle \text{cov},\ \text{oog},\ \text{bg},\ \text{pre},\ \text{mc},\ \text{edit} \rangle \tag{3}$$

defined in §5.3. Note that $C(u)$ appears on both sides: this is the formal statement of "demand and check share one source."

Fig. 1 shows the pipeline: graph → contract compilation → generation → rendering → validation against the same contract. One loop exists—pages that fail validation may be regenerated—and its boundary is fixed: **the loop may change slides, but never the graph and never the contract**. Whether to regenerate is a teacher's decision. Were validation output allowed to reshape the contract, acceptance would degrade into an objective (§7.1).

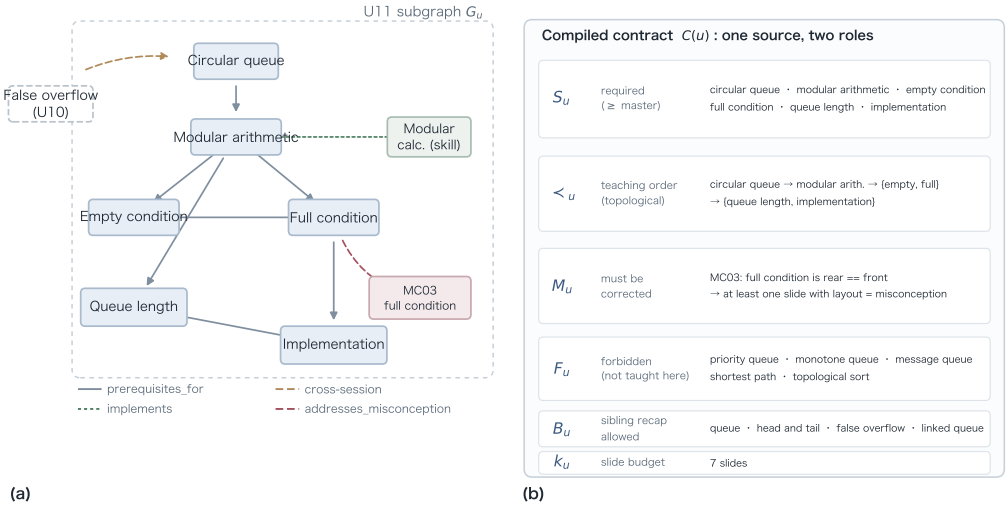


Figure 2: From subgraph to contract. (a) The actual U11 subgraph in the released graph file: six concepts and one skill, seven intra-session prerequisite edges, one cross-session prerequisite edge from U10, one `implements` edge and one `addresses_misconception` edge. (b) The contract $C(u)$ produced from it by `build_constraints`; the validator subsequently reads the same object.

4. Method

4.1. Graph construction

4.1.1. Nodes and edges

We keep the pedagogical subset of K12-KGraph's node skeleton and rename it to match course structure: its `Book`/`Chapter`/`Section` become our `Course`/`Module`/`Unit`, and its `Experiment` is rendered as `Lab`. `Misconception` is promoted to a first-class node, with attributes `statement` (the erroneous claim itself, which must be concrete and decidable), `correction`, `frequency` (low, common, high) and `evidence` (exam error rate, assignment statistics, in-class questioning).

Two edge types are added:

- `implements`: Concept or Skill → executable code fragment. In the current system it supplies the evidence anchor the prompt cites for a concept and makes the intended code artifact addressable. No headline metric reads it, and the only execution that touches it is the bounded probe of §7.6; it is not otherwise a scoring dimension.

- `addresses_misconception`: Unit, Lab or Concept → Misconception, annotated with a strategy (counter-example, code demonstration, boundary test). This edge decides **which session is responsible** for correcting a given misconception, preventing both duplication across sessions and orphaned misconceptions.

4.1.2. Synonyms

Every concept may carry aliases. This is not a patch for evaluation but legitimate graph content: in class the teacher says "judging full," the textbook writes "the full condition," and the code reads `(rear+1) % maxsize == front`; all three denote one entity. Without aliases the graph cannot be aligned with real teaching language. Aliases also resolve a fairness problem in evaluation, as discussed in §7.1.

4.2. The graph is checked, not merely loaded

The contract is only as trustworthy as the graph that produces it, so the graph is validated on ten checks (twelve codes; checks 5 and 10 emit two each) every time it is loaded. There is no sampling step to weaken this: a load that raises stops the pipeline.

Checks 1–6 belong to the schema: no dangling edge, no prerequisite cycle, every knowledge point grounded in a unit, evidence on every semantic edge, three to eight concepts per unit with at least one at mastery level, and no duplicate name within a module. Checks 7–10 are the ones this paper depends on, because they test whether the graph's own contract ingredients contradict one another rather than merely whether the file is well formed:

- $\mathbf{F_u} \cap \mathbf{R_u} = \emptyset$. A term may not be simultaneously mandatory (a mastery-level concept of this session) and forbidden (a declared out-of-scope term). If it were, the contract would ask the model to cover and to avoid the same thing, and a coverage shortfall would be an artefact of the graph.
- $\mathbf{F_u} \cap \mathbf{B_u} = \emptyset$. A term may not be forbidden here and admissible as a one-sentence recap because a sibling session of the same module taught it. B_u is the sibling set `build_constraints` actually emits, so the check tests the contract as shipped, not an idealisation of it.
- **No orphan misconception.** Every misconception node must be the target of at least one `addresses_misconception` edge. An orphan can never enter any contract: the metric would have nothing to score, and the 0.556 reported for the control conditions would be a property of the graph rather than of generation.

- `implements` **is well typed and grounded.** The edge must point at an executable artefact (Skill, Lab, Exercise, Concept) rather than at a misconception or a unit, and its source must be taught in at least one session.

The shipped graph (34 nodes, 75 edges) returns zero findings on all ten checks, and `pytest` asserts it. Zero findings is weak evidence on its own, because a checker that never fires cannot be distinguished from one that cannot fire. Each of the twelve codes is therefore exercised by mutation: one minimal defect is injected into a copy of the graph and we record whether the owning code responds. All twelve fire on their own mutant, and a single mutant triggers a second code—an ungrounded skill also violates check 3—which is the correct verdict rather than interference, so the checks are specific as well as sensitive. Table S2 in the supplementary material lists every injected defect; the harness is `paper/graph_checks.py`, and five unit tests pin the four new checks to mutants so that a regression fails the suite.

What this buys is a weaker and more defensible claim than "the graph reasons." The graph does not infer anything: the topological order, the required set and the misconception assignment are computed by deterministic graph algorithms, and their correctness is established by the checks above, not by the model. The reasoning that a knowledge-based system is supposed to contribute happens earlier—in how the course was decomposed into sessions, concepts and misconceptions, and in the decision that a session's obligations are a function of that decomposition. Section 7.1 is explicit about which of the resulting metric defects the graph can detect and which it cannot.

4.3. Constraint compilation

`build_constraints` turns the subgraph into a `UnitConstraints` object. Every field below is a function of the graph; none is entered per session:

1. **Required/optional split.** Sort by `importance`; those at or above "master" enter R_u, the rest O_u.
2. **Topological order.** Run Kahn's algorithm over S_u; ties are broken by declaration order, so the result is deterministic. If a cycle reaches the compiler it falls back to declaration order and warns, but this branch is defensive only: a cyclic graph is rejected at load time (§3.1) and never gets here.
3. **Misconception assignment.** Collect the misconception nodes attached by `addresses_misconception` edges.

4. **Sibling sessions.** Concept names of other Units in the same Module, minus those already in this session.
5. **Evidence anchors.** Collect `evidence` along incoming `prerequisites_for`, `implements` and `verified_by` edges, and render them as *teacher-facing* verification material, explicitly labelled "not generated content" so that the model does not copy the provenance into the slides as body text.

The compilation output is a *deterministic string*: the same graph always yields the same prompt block, which is a precondition for reproducibility.

4.4. Generation

The prompt is assembled from four segments—a fixed skeleton, the contract block, a JSON contract, and a list of hard rules—and the experimental conditions of §5.2 replace only some of them. The assembled prompt for session U11 is rendered in full in Appendix Appendix A, and one raw response with its validator report in Appendix Appendix B; neither is summarised away, because the prompt is part of the method.

Output is constrained to a single JSON object, each slide carrying `layout` (one of seven), `title`, `bullets`, `code`, `notes`, `concept_ids` and `background`. Forcing JSON rather than free-form layout is a deliberate choice: layout selection is itself an instructional-design decision, and delegating it to the model makes both editability and slide budget uncontrollable.

The caps on bullets and characters are conventions rather than derived constants, though not arbitrary ones: cognitive load theory predicts that dense, split-attention material imposes extraneous load competing with the schema the lesson is building [32], and the same concern returns from the learning-sciences side as an argument that benchmark scores are the wrong criterion for educational AI, because what matters is whether the artefact supports the knowledge component being taught [5]. A four-bullet page is a rough operationalisation of both, and we treat it as a structural constraint the validator can check rather than as a claim about pedagogy.

Two engineering realities, observed rather than assumed: JSON mode is not a validity guarantee—roughly one call in ten returns a trailing comma, or blank with `finish_reason == "stop"`—so the pipeline repairs conservatively (strip trailing commas, close truncated brackets) and escalates by re-prompting with the concrete parse error only when repair fails, since blind retries reproduce the same error at the same cost; and the model emits `reasoning_tokens` on every call, which contaminates a chain-of-thought

baseline because the model is already reasoning. We record the setting and provide a switch to disable it.

4.5. Rendering

`python-pptx` emits 16:9 fully native shapes with no rasterised text. `editability` is defined as the ratio of native editable shapes to all shapes; it is 1.0 for every condition here. This dimension does not discriminate between methods; it exists to exclude "screenshot-style generation," whose value for a teacher who needs to edit is zero.

For Chinese text the East-Asian typeface attribute must be set explicitly, or presentation software falls back to a Latin font and the glyphs are corrupted.

4.6. Validation

The six metrics are defined in §5.3. Three implementation choices are deliberate.

- **Chinese has no word boundaries.** The term for *queue* matches inside the term for *circular queue*. All counting therefore uses per-line net occurrences: raw count minus occurrences covered by longer terms *within the same line*. Subtracting over the whole document rather than per line is a real bug: an independent occurrence on slide 2 is otherwise cancelled by a compound term on slide 4.
- **Triple-criterion misconception detection.** Structural anchor (does the misconception slide reference a graph-linked concept), then English-identifier overlap (`rear`, `list`), then Chinese character bigram overlap (threshold 0.35). Purely lexical matching misses cases where the model paraphrases the erroneous claim in its own words; we observed a generated correction slide rejected because its phrasing differed from the graph's.
- **Order by first textual position**, see §5.3.

5. Experiments

5.1. Course and data

The target course is *Python Data Structures*, a higher-vocational programme in Artificial Intelligence Technology Application with 136 contact hours. The graph scope is Module M4, "Queues", comprising three sessions (U10, concepts and false overflow; U11, circular queue; U12, linked queue and

Table 2: Six experimental conditions. Prompt lengths are measured on session U11, in characters.

Condition	Course/ session	Page budget	Concept names + definitions	Topological order	Misconceptions	Forbidden terms
`full` (ours)	✓	✓	✓	✓	✓	✓
`rag_mc`	✓	✓	✓	–	✓	–
`rag`	✓	✓	✓	–	–	–
`placebo`	✓	✓	–	–	–	–
`cot`	✓	✓	–	–	–	–
`zero_shot`	✓	✓	–	–	–	–
Prompt length, U11	1997 / 1122 / 997 / 2195 / 625 / 585					

introducing breadth-first search), with 34 nodes, 75 edges and four misconceptions. The prerequisite chain in this module is clean (queue → sequential queue → circular queue → linked queue), whereas deep-learning courses have mutually referencing concepts that easily form cycles and are therefore unsuitable as a first validation scenario.

The goal is to establish *whether and along which dimensions* graph constraints help, not to maximise a course-wide number. Three sessions cover three qualitatively different cases: concept-dense (U11 has seven required concepts), skill-landing (U10 contains a skill that must be implemented by hand), and application-introduction (U12 requires a comparative selection against prior material).

5.2. Conditions

Six conditions carry strictly increasing information, each step removing one structural element (Table 2).

- `rag` − `zero_shot` measures the contribution of *content retrieval*.
- `full` − `rag` measures the contribution of *structural constraints*.
- `rag_mc` is a control built specifically for the misconception claim: the same misconception text is supplied as a flat passage, but without ordering and without layout requirements. If `full` still wins on the misconception metric, the gain comes from *structuring the text to the point where it constrains layout*, not from *the text appearing in the prompt*.
- `placebo` is a control built specifically for the alternative explanation "the prompt is simply longer": the prompt is *length-matched* to `full` (in fact

slightly longer), but padded with generic slide-writing advice unrelated to this course. If the advantage of `full` were prompt length, the placebo should catch up. It does not (§5.4).

What `cot` actually adds. `cot` and `zero_shot` occupy the same row of Table 2: both carry the course name, session name, page budget, output schema and baseline rules, and neither carries a concept list. The difference is one added rule—before writing a page, state in its `notes` where it sits in the lecture and how it connects to the previous page. That is the cheapest stand-in for chain-of-thought prompting [33, 34] and not the same thing: the model reasons about the *deck*, not about the course. We report it as a planning baseline, not as CoT over course structure, which would require the structure under test.

A note on the name `rag`. The two retrieval conditions run no retriever. Their passage set is the definition of every concept in the session, read from the graph and inlined whole: no index, no chunking, no top-k, no similarity search. The identifier is kept because it names the condition in the released run files, but the mechanism is *oracle passage supply* with recall 1 by construction. This makes the baselines stronger than a deployed RAG system, not weaker, and it means our comparison is not against RAG as practised: what it isolates is the *form* in which correct content arrives. An end-to-end retrieval baseline with a corpus, an embedding model and a measured recall is future work, and it is the honest way to ask whether the graph beats retrieval rather than beats flat text.

Three ablations retain the complete contract and rule wording of `full`, deleting exactly one data block: the ordering block with its rule 3, the misconception block with rule 5, and the forbidden-term block with rule 6. Attribution is in Table 5; the full per-metric matrix is Table S1 of the supplementary material. The design follows one rule—the rule text must be removed together with the data, since keeping "you must follow the given order" while deleting the order would force the model to guess and the ablation would no longer be clean. A unit test guards this.

5.3. Metric definitions

The six metrics are listed in Table 3.

Three definitional decisions determine the conclusions and therefore require explicit justification:

Table 3: Metric definitions. Arrows indicate the desired direction.

Metric	Definition	Direction
`coverage_rate`	fraction of required concepts actually taught	↑
`oog_rate`	(out-of-graph IDs + forbidden-term hits) / (covered + out-of-graph)	↓
`background_rate`	characters devoted to sibling-session concepts / total characters (cap 25%)	↓
`prerequisite_violation`	violated concept pairs / all concept pairs	↓
`misconception_coverage`	misconceptions corrected / misconceptions linked to the session	↑
`editability`	native editable shapes / all shapes	↑

1. **Coverage counts body slides only.** Title, objectives and summary slides list every concept by definition; counting them as "taught" lets a deck consisting of a table of contents score full marks. Every condition in every arm emits all three pages, so the exclusion is symmetric across the comparison—but it is a *convention*, not a fact about teaching, and §5.3 reports what happens when it is relaxed.
2. **Coverage matches on name and aliases.** Otherwise the `full` condition, which copies source names, enjoys a structural advantage (§7.1).
3. **Order is judged by first textual position.** The model's `concept_ids` are self-reported, and the three baselines were never told the field exists. Scoring on self-reports means a completely empty answer receives a zero violation rate.

Is the preview-page convention doing the work?. The first decision is the one a reader should distrust, so we recomputed coverage for all 54 main runs under two relaxations: keep the title slide, and keep every slide. Only two runs move when the title slide is admitted—one `full` and one `cot`—the ranking of the six conditions is unchanged (Spearman $\rho = 1.00$; $\rho = 0.94$ once objectives and summary are also admitted), and the headline contrast against `rag_mc` widens rather than shrinking ($+0.283 \rightarrow +0.302$). Admitting objectives and summary lifts the retrieval baselines most, because those pages restate retrieved text. The convention is therefore not carrying the conclusion—although, as §5.8 concedes, the one negative case we discuss does depend on it.

The aliases decision points the same way, and more sharply. Recomputing

coverage with name-only matching, so that a deck writing the bare verb where the graph names the full operation pair is marked as having skipped the concept, drops baseline coverage to 0.325–0.389 while `full` falls only to 0.963, because `full` was handed the canonical strings and copied them. Thirty-one of the 54 runs move, the largest single drop from 1.000 to 0.143. Alias matching therefore *narrows* the reported gap: the conservative number for `full` against `rag_mc` is +0.283, and the gap a stricter matcher would produce is +0.574. This is the opposite of a defect that manufactures the result—it is a correction that runs against the hypothesis, which is why we keep it and why the reported advantage should be read as a lower bound rather than a best case.

5.4. Results

Each condition runs three sessions × three resamplings (nine generations); each ablation arm runs three sessions × two (six). The ablation file also holds six re-runs of `full`, which is where the total of 78 comes from—54 + 18 + 6. We report 54 and 18 as the experimental counts and treat 78 as the number of API calls, not of independent units of evidence. Runs are nested in three sessions, so the intervals below resample runs within a condition and the three session means are printed beside each; no mixed model is fitted, because three clusters cannot support one.

Pass rate is the share of runs with no *error-level* finding—a missing required concept (`MISSING_CONCEPT`) or a prerequisite violation (`PREREQ_VIOLATION`); out-of-graph content and a missed misconception are warnings and do not fail a run. That is why `full` passes six of nine while missing no misconception at all, and why we do not call the system "always usable": a third of its runs still need a human.

Five observations follow, ordered by importance.

(1) Coverage: 0.98 against 0.61–0.70, but the gain is not uniform across sessions. The five controls lie within 0.09 of one another (0.614–0.698), all with large dispersion (0.19–0.30); `full` reaches 0.981. Run-level intervals do not overlap—$[0.944, 1.000]$ against $[0.524, 0.878]$ for the nearest control, `rag_mc`—a difference of +0.283 $[+0.101, +0.471]$ with Cliff's $\delta = +0.60$: a randomly drawn `full` deck outscores a randomly drawn `rag_mc` deck with probability 0.80.

That interval resamples runs and so treats nine correlated draws as independent. Resampling whole sessions—three clusters, all 27 resamples

Table 4: Main results ($n = 9$ per condition), mean (standard deviation). Cov.: coverage; OOG: out-of-graph; Bg.: background; Ord.: prerequisite violation; Miscon.: misconception. Best value per column in bold. Pass rate is the share of runs with no error-level finding, i.e. no missing concept and no order violation; warnings do not fail a run (§5.4). Editability is 1.000 for every condition and is therefore omitted.

Condition	Cov. ↑	OOG ↓	Bg. ↓	Ord. ↓	Miscon. ↑	Pass rate
`full` **(ours)**	**0.981** (0.056)	**0.000**	0.027 (0.016)	**0.020** (0.045)	**1.000** (0.000)	67%
`rag_mc`	0.698 (0.295)	0.111 (0.220)	0.027 (0.016)	0.007 (0.022)	0.556 (0.527)	22%
`rag`	0.659 (0.248)	0.111 (0.220)	0.032 (0.014)	0.030 (0.089)	0.222 (0.441)	0%
`placebo`	0.675 (0.266)	0.167 (0.250)	0.040 (0.018)	0.096 (0.131)	0.333 (0.500)	0%
`cot`	0.614 (0.185)	0.019 (0.056)	0.040 (0.016)	0.089 (0.183)	0.389 (0.486)	0%
`zero_shot`	0.643 (0.248)	0.019 (0.056)	0.040 (0.011)	0.037 (0.089)	0.222 (0.441)	0%

enumerated—widens it to $[-0.056, +0.667]$, and the per-session contrast shows why: $+0.238$ on U11, $+0.667$ on U12, -0.056 on U10. **The coverage advantage is carried by two of three sessions and is absent in the third.** We give both intervals—the run-level one because it is what nine runs per condition support, the clustered one because it is the honest uncertainty—and the claim surviving both is narrower than "coverage rises from 0.61 to 0.98" (§7.2).

The decisive evidence against the length explanation is the placebo: its prompt is comparable in length to `full` (2195 against 1997 characters on U11, in fact longer) but its content is generic writing advice, and its coverage is 0.675 $[0.505, 0.833]$, indistinguishable from zero-shot.

Concept-level pairing. Two intervals over three clusters leave the coverage claim weaker than the numbers suggest, so we ask the question one level down, where the contract is actually written. Scoring every required concept on every run gives $16 \times 3 = 48$ paired observations per baseline. Against `zero_shot` the discordant cells split 14 in favour of `full` and 1 against—a paired risk difference of $+0.271$ with McNemar $p = 9.8 \times 10^{-4}$; against `rag_mc`, the control that receives the same misconception text, 11 to 1 ($+0.208$, $p = 6.4 \times 10^{-3}$). All five baselines give the same sign and all sit below $p = 0.01$ (Table S5). The pairs are not independent—sixteen concepts nested in three sessions and resampled—so we do not read those p values as population inference; what they add is that the advantage is not carried by one or two concepts. Leaving each concept out in turn moves the per-session gap by at most 0.056 (U10 -0.067 to 0.000, U11 $+0.111$ to $+0.278$, U12 $+0.500$ to

+1.000). The weak session survives the same treatment: U10 is still the only session whose sign can flip, which is why the clustered interval above still includes zero.

(2) Misconception: the mean is the lesser finding; the disappearance of variance is the greater. `rag_mc` receives the same misconception text and reaches 0.556, but with interval $[0.222, 0.889]$ and standard deviation 0.527: it succeeds fully in about half of nine generations and misses entirely in the rest. `full` reaches 1.000 with degenerate interval $[1.000, 1.000]$ and zero dispersion; the difference is $+0.444$ $[+0.111, +0.778]$. Unlike coverage, this survives clustering—$+0.333$ on U10, $+0.333$ on U11, $+0.667$ on U12, so the cluster interval $[+0.333, +0.667]$ excludes zero. **Of the two headline results, the misconception one is the one that holds up.**

What it supports is narrower than "the graph teaches better". `full` differs from `rag_mc` in several ways at once—ordering block, forbidden list, layout rules, and a page of a particular kind. The defensible reading is that **a contract specifying where corrective content must appear gets it to appear reliably, whereas the same content as flat prose does not**; whether the active ingredient is the dedicated page, the surrounding structural demands, or the mention of a required slot is not identified by this design (§7.2).

(3) Ordering: where the representation runs out. `full` violates order at 0.020 $[0.000, 0.050]$, below the placebo (0.096) and `cot` (0.089), but `zero_shot`, told nothing about order, sits at 0.037 $[0.000, 0.096]$: the difference is -0.017 $[-0.086, +0.037]$, an interval too wide to exclude a small effect in either direction. The ablation does not narrow it, because the arm we built to remove the order does not remove it. The required-concept block is emitted in topological order—the same order the deleted block restates—so `no_order` still tells the model what comes first, and six runs of a duplicate cannot price the original.

This is the dimension §1.5 marked in advance, and the result matches the prediction rather than revising it: no baseline is disadvantaged by the order it was not given, and no condition here withholds the order it was given. **Ordering is therefore neither refuted nor confirmed—it is the dimension on which this representation stops being able to separate the requirement from the content that satisfies it, and we classify it as a boundary rather than price it as an effect.**

(4) Out-of-graph: `rag` and `rag_mc` are worse than zero-shot. Both reach 0.111 and the placebo 0.167, whereas `zero_shot` sits at 0.019 and `full` at 0. Telling the model "this session covers these points" leads it to bring in adjacent advanced material—mentioning priority queues in a session on queues—because the prompt demarcates a scope without demarcating a boundary. `full` reaches zero only through the forbidden-term list, an explicit negative example, as the ablation confirms.

(5) Attribution: each knowledge component buys the metric it governs. Table 5 deletes one block at a time and re-measures every metric, so the ablation reads as an attribution rather than as a second set of condition means. The misconception slot is worth 0.833 of misconception coverage on its own, and its removal costs 0.056 of coverage as well, because corrective content that names the concept and its failure mode organises the key points by the way. The forbidden-term list is worth 0.083 of out-of-graph control and moves nothing else—a clean single-responsibility attribution. The ordering block moves its own metric and no other, which is the exception already discussed: the deleted line duplicates an order the prompt still carries, and the 0.032 it costs is one run out of six (U11, resampling 1, four inversions among twenty-one concept pairs, 0.191, amortised over the batch). **The positive gains in this paper are attributable to named components rather than to the pipeline as a whole: each block moves the metric it governs and leaves the others at their floor or ceiling, with one spillover—the slot also carries** 0.056 **of coverage—and the one arm that moves nothing is the arm whose manipulation leaves its target in place.**

`full` consumes 1064 input and 4079 output tokens (2708 of them reasoning) at 20.8 s end to end, against 347, 2330, 984 and 12.4 s for `zero_shot`: about 1.75 times the output, the surplus being the contract block (roughly 500 input tokens) and fuller speaker notes—the page count is fixed by `slide_budget` and met in every run. The placebo is the informative contrast on input: 1252 tokens, more than `full`, with only 111 reasoning tokens. Per deck the cost stays in the cents range, negligible against preparation time, but for course-wide batch generation the 1.75× gap is worth engineering away, for example by caching contract blocks per session.

5.5. A second backbone replicates the contrast

Every number above comes from one vendor's model, which leaves the obvious objection: a contract might be compensating for one model's prompt

Table 5: Attribution by ablation: each block is deleted alone and all four metrics re-measured in the same batch ($n = 6$ per arm). Δ is the arm minus `full`; a block's contribution is the movement in the metric it governs. Secondary movement above 0.04 is shown as a side effect. The ordering row is a null manipulation rather than a measurement of the constraint (§1.5); the full arm of this batch carries one order-violating run, which is why every other arm reads 0.000. Mean (standard deviation) over resamplings.

Block removed	Metric	`full`	Without	Δ
Misconception slot	Misconception ↑	1.000 (0.000)	0.167 (0.408)	−0.833
Misconception slot	Coverage ↑ (side)	1.000 (0.000)	0.944 (0.086)	−0.056
Forbidden list	Out-of-graph ↓	0.000 (0.000)	0.083 (0.129)	+0.083
Ordering block	Order viol. ↓	0.032 (0.078)	0.000 (0.000)	−0.032

tuning rather than supplying knowledge. We therefore re-ran the four claim-bearing conditions on Kimi K3—same prompts, sessions and validator, six runs each, five for zero-shot, which is reported as missing rather than refilled (Table S4 in the supplementary material). That vendor pins sampling, so the replication runs at temperature 1.0 against 0.3; within-model contrasts are unaffected, cross-model levels are not compared.

The sign replicates on both metrics that carry the paper. Misconception coverage falls from 1.000 under `full` to 0.667 under `rag_mc` (−0.333, against −0.444 on DeepSeek) and to 0.333 for the length-matched placebo; coverage falls from 1.000 to 0.734 (−0.266, against −0.283). More telling than either mean is the variance: `full` returns 1.000 in every run under both models, while `rag_mc`'s standard deviation is 0.516 on Kimi against 0.527 on DeepSeek. **What replicates is not an effect size but a property: a contract that names the slot fills it every time, and the same text without a slot fills it two times in three.**

Two things do not replicate, and they bound the claim. The gain is smaller on the second model (0.333 against 0.444), which is what a stronger model should produce if the contract compensates for a weakness rather than teaching something the model cannot do; and on U11 every condition scored 1.000 on Kimi, zero-shot included, so the session that carries the effect under one model carries none under the other. Six runs also leave the pass-rate gap (67% against 17%) without a usable interval. **The replication rules out a single vendor's artefact and supports nothing about model**

Table 6: Second course (*C++ Programming*, arrays chapter), three conditions × two sessions × two resamplings ($n = 4$ per condition), a different backbone and endpoint from Table 4. Mean (standard deviation). Generated by `paper/tables.py -course-b outputs/bench_cpp`.

Condition	Coverage ↑	Out-of-graph ↓	Order viol. ↓	Misconception ↑
full (ours)	0.800 (0.163)	0.000 (0.000)	0.042 (0.083)	0.625 (0.479)
rag_mc	0.600 (0.231)	0.000 (0.000)	0.017 (0.033)	0.375 (0.479)
zero_shot	0.350 (0.191)	0.125 (0.250)	0.042 (0.083)	0.000 (0.000)

families; one more model would not change that.

5.6. A second course: the direction replicates, the reliability belongs to the model

Every result so far comes from one course, so the natural objection is that the contract does nothing a competent teacher's prompt would not do on a different syllabus. We therefore replicated the two claim-bearing conditions on a second course: the arrays chapter of *C++ Programming*, taught in the same department to a different cohort (class 4 of the 2024 Animation Production Technology intake, four contact hours per session), with a graph built from that course's own lesson plan—the same object type, different knowledge, different programming language.

Two sessions, three conditions (`full`, `rag_mc`, `zero_shot`) and two resamplings: twelve runs. Two things differ from the main study, and both are stated rather than smoothed over. The backbone is a different model on a different endpoint (`agnes-2.5-flash`, temperature 0.3), chosen because the primary key was out of balance on the day of the run. And the course's misconceptions come from the lesson plan's difficulty list and its reflection section rather than from exam error statistics: they are instructor-documented difficulties, not measured frequencies. The replication therefore tests course and backbone jointly, and its magnitudes must not be read against Table 4.

The direction survives. Coverage is 0.800 (0.163) against 0.350 (0.191) for zero-shot and 0.600 (0.231) for the text-matched control—gaps of +0.450 and +0.200, larger than the +0.283 of the first course because the weaker backbone starts lower. Misconception handling is 0.625 (0.479) against 0.000 (0.000) for zero-shot and 0.375 (0.479) for the control given the same misconception text. Out-of-graph behaviour reproduces the first course's pattern

too: zero-shot imports material from later chapters (0.125), the contract does not (0.000). Both claim-bearing contrasts keep their sign on a different course, a different language and a different backbone.

The second moment does not. In the first course the slot produced 1.000 ± 0.000; here it produces 0.625 ± 0.479—two runs of four fill the slot and two do not, and the matched control has the same dispersion. The contract still moves the mean, and still beats the control that carries the same text, but it does not make the outcome certain. Two explanations are available and this design does not separate them: the backbone is weaker on the task (its zero-shot coverage is 0.35 against 0.64, and it is the only condition here that imports out-of-graph content), and the misconceptions are documented difficulties rather than measured ones, so the prompt states what students get wrong without stating how often. We report both and adopt neither.

Ordering is the third thing to check, because that is where the scope claim makes a prediction. Violations are 0.042 (0.083) for `full` and 0.042 (0.083) for `zero_shot`, with 0.017 for `rag_mc`: no contrast, on a course whose textbook order is canonical as well. The dimension that cannot be withheld fails to separate conditions here too, and for the same structural reason (§1.5)—the second independent reading of the same boundary.

What the second course buys is therefore narrower and more useful than "it generalises". The *direction* of both claim-bearing contrasts is course-independent. The *reliability* claim is conditional on a backbone able to execute a structural instruction: a contract can require a slot, but only a model that follows structural requirements will fill it every time. That condition is not visible inside one course, and it is the kind of statement a reader can check against their own model.

Which of the two changed. The second course moves syllabus, language and endpoint at once, so the failure above has two candidate explanations and its own design cannot choose between them. One more run can. Holding the course, the graph, the prompts, the validator and the metric code fixed—the first course, unchanged in every respect—we re-ran the same three conditions on the backbone the second course used (`agnes-2.5-flash`): three sessions, two resamplings, 18 runs, Table 7 row 3. The slot's advantage disappears there too, and in the same way: 0.833 ± 0.408 against 0.833 ± 0.408 for the control that receives the same misconception text, a difference of 0.000 where the same course produced +0.444 under DeepSeek and +0.333 under Kimi. Only the endpoint changed, so **the precondition tracks the model rather than the course**; the second course's failure was most plausibly this

Table 7: The same two paired contrasts on every (backbone, course) cell we have. Rows 1–3 hold course, graph, prompts, validator and metric code fixed and vary only the endpoint; row 4 holds the endpoint fixed and varies the course. Δ is `full` minus `rag_mc`; the last column is the distribution the reliability claim is about. Generated by `paper/tables.py -sweep`.

Backbone	Course	Δ coverage	Δ misconception	`full`'s misconception
DeepSeek V4	Python	+0.283	+0.444	1.000 (0.000)
Kimi K3	Python	+0.266	+0.333	1.000 (0.000)
agnes-2.5-flash	Python	+0.052	+0.000	0.833 (0.408)
agnes-2.5-flash	C++	+0.200	+0.250	0.625 (0.479)

behaviour, encountered on a different syllabus.

That sentence is the strongest inference in the paper and it should not be read as more than it is. Table 7 is four cells with two resamplings each, compared descriptively; nothing here is an interaction test, and a three-cluster interval on any of those differences would include zero. What the comparison establishes is that the contrast is *absent* when the endpoint changes and everything else is held fixed—it removes the syllabus as the explanation without identifying which model property took its place. We therefore report a pattern over four cells, consistent with a backbone that does not fill a required slot reliably, and leave the mechanism open.

Two details keep this from collapsing into "the model decides". First, the forbidden-term requirement still binds on the same runs—out-of-graph content falls from 0.097 under the text-matched control to 0.000, exactly the pattern of the first course under DeepSeek—so what this backbone does not do is fill a required *page*; it is not indifferent to structure in general. Second, the coverage margin over the text-matched control falls to +0.052 on this backbone while the margin over zero-shot stays at +0.401: on a model that already reads the supplied text carefully, a contract that supplies the same text buys less. Neither observation is a mechanism—two runs per cell make them patterns, and we report them as patterns, not as a capability threshold.

5.7. Teacher blind review

The six metrics measure conformance to the contract, not teaching quality (§7). To check that the two agree, ten decks—three from each of `full`, `rag_mc` and `rag`, one from `placebo`—were scored independently by three teachers

who did not know which condition had produced a deck. Each deck was rated on five five-point items and on an overall score, defined as the unweighted mean of the five; Table 8 reports the result, and the raw ratings are released with the repository.

Protocol.. Decks were exported to PDF with speaker notes removed and file names replaced by opaque identifiers, and presented in one folder in an order fixed before scoring, with no condition label and no run identifier. The raters are three teachers of the same department who teach this course or its prerequisite. They scored in one sitting, independently, without discussing items or comparing decks, and were told only the course, the session and the item wording—not the research question, not the automatic metrics, and not that the decks came from more than one method. Items are anchored 1 (absent) to 5 (fully adequate) and reported verbatim in Table 8; the overall score is their unweighted mean, which `human_eval.py` verifies to 10^{-4}. No rater was paid and all three consented. We ran no training or calibration round, which is a real limitation: without one we cannot separate disagreement about the rubric from disagreement about the decks, and the pedagogical-logic ICC of 0.53 may reflect either.

Table 8: Teacher blind review on a five-point scale, mean (standard deviation) over ratings. Three decks per condition except `placebo` ($n = 1$ deck, descriptive only). ICC(2,3) is the agreement of the average of three raters. Content accuracy has no between-deck variance to model ($F(9, 18) = 1.00$), so its ICC is zero by construction rather than by disagreement.

Item	`full`	`rag_mc`	`rag`	`placebo`	ICC(2,3)
Syllabus coverage	4.78 (0.44)	3.56 (0.53)	3.33 (0.50)	3.67 (0.58)	0.80
Misconception correction	4.89 (0.33)	3.22 (1.48)	2.11 (0.60)	2.67 (0.58)	0.91
Pedagogical logic	4.56 (0.53)	3.78 (0.44)	3.67 (0.50)	3.67 (0.58)	0.53
Content accuracy	4.56 (0.53)	4.22 (0.44)	4.11 (0.33)	4.00 (0.00)	0.00
Classroom readiness	4.67 (0.50)	3.33 (0.50)	3.11 (0.33)	3.00 (0.00)	0.91
Overall	4.69 (0.28)	3.62 (0.47)	3.27 (0.24)	3.40 (0.20)	**0.92**
Decks	3	3	3	1	
Ratings	9	9	9	3	

Agreement is high where it matters. Averaging three raters gives an overall ICC(2,3) of 0.92 (95% CI $[0.69, 0.96]$, bootstrap over decks), strongest on the two items the manipulation targets—misconception correction (0.91)

and classroom readiness (0.91)—with coverage next at 0.80. The two low values are informative. Content accuracy has no between-deck variance to model (every condition averages at least 4.0, $F(9, 18) = 1.00$), so its ICC is zero by construction rather than by disagreement; pedagogical logic (0.53) is where raters diverge most, which is plausible, since judging sequencing from slides requires knowing what the course already covered and the raters were not told.

The ordering is unambiguous: `full` averages 4.69 (0.28) against 3.62 (0.47) for `rag_mc`, 3.40 (0.20) for `placebo` and 3.27 (0.24) for `rag`, and every `full` deck (4.53–4.80) outscores every `rag_mc` deck (3.27–4.00). Complete separation of three against three gives a two-sided permutation $p = 0.10$, the smallest attainable at this sample size, so we read the human data as *consistent with* the automatic results rather than as confirmation at conventional significance.

The agreement is more specific than a shared ranking. Rescaling the misconception item to $[0, 1]$ gives 0.97, 0.56, 0.42 and 0.28 against automatic values of 1.000, 0.556, 0.333 and 0.222: identical rank order, gap at most 0.09. Content accuracy stays at or above 4.0 everywhere with a spread of 0.56 points, echoing the automatic finding that the baselines are accurate but incomplete. Teachers localise the difference where the metrics do—not in whether the slides are right, but in whether they teach what this class needs.

Three limits apply. The raters are one cohort at one institution; three decks per condition permit description but not inference; and the `placebo` arm contributes one deck, so its column carries no weight.

5.8. Qualitative observations

A positive case: `full` on U12. U12 is the hardest session: it requires choosing between a circular and a linked queue, a judgement task with no single correct answer, and one required concept ("application scenarios of queues") is abstract and easily reduced to platitudes. All three `full` runs reached coverage 1.000 and each produced a genuine comparison slide (predictable capacity and frequent queries favour the circular queue; variable length and frequent enqueue–dequeue favour the linked queue), handling the breadth-first-search misconception on a dedicated counter-example slide. Coverage for the five baselines on the same session is **0.333**: they cover only "linked queue" and dispose of selection and scenarios in one clause.

A negative case: `full` fails too. In the third resampling of U10, coverage is 0.833, missing the concept "queue" itself. Reading slide by slide: the title slide states that a queue is a linear list inserting at one end and deleting at the other, but the first body slide is titled "head, tail and the sequential queue," and the deck never defines a queue separately again. The model assumed students would remember the definition from one sentence on the cover, and nothing in the following five slides reinforces it.

What kind of failure this is matters. Whether a definition on the title slide counts as teaching is a pedagogical convention, and the omission disappears if the title slide is admitted (§5.3: coverage rises to 1.000); anyone who thinks a cover definition suffices is entitled to read this run as a pass. What the metric records is narrower: the definition is stated once, in a position the validator does not treat as instruction, and never returned to. The validator flagged it, which is the point of closing the loop—the method does not guarantee correctness, it guarantees that **errors are not silently passed over**, and it leaves the disposition to the teacher.

A second failure is convention-independent and therefore more instructive: the second resampling of U10 contains two prerequisite inversions, the model teaching the sequential queue first and returning to head and tail afterwards. On the metrics this is a 0.133 violation rate; in the classroom it means students first encounter the head and tail indices without knowing what they denote. The topological order was in the prompt and the model violated it anyway, which is why we insist on post-hoc validation rather than assuming that saying it implies doing it.

6. Discussion

6.1. Where the graph helps and where it does not

Rearranging the six metrics by the question "does the graph make an irreplaceable contribution" sorts them into four groups. *Irreplaceable*: misconception handling (1.000 ± 0.000 against 0.556 ± 0.527 with the same text) and coverage completeness (0.981 against 0.61–0.70, with the length-matched placebo excluding prompt length). *Partial*: scope control, which rests on the forbidden-term list as an explicit negative example, as the ablation confirms ($0.000 \rightarrow 0.083$). *Beyond this representation*: prerequisite order (0.020 against 0.037 for zero-shot, with the ablation running the other way, for the structural reason given in §1.5). *Orthogonal*: editability, 1.000 everywhere and determined by the renderer.

The first two groups are a result about knowledge; the third is a result about representation, and the two should not be read the same way. **The graph supplies not disciplinary knowledge but the layer of institutional convention around it.** A strong model already handles textbook content and textbook order; which concepts count as "must be finished in this session," where this cohort errs, and what belongs to the next session appear in no pretraining corpus, and once supplied the model uses them stably. Order is the exception that tests the rule: a model needs no help with textbook order, so the only order worth constraining is institutional—and that is precisely the kind the interface cannot deliver on its own.

The pattern across the four groups is not that graphs help a little everywhere. It is that **actionability follows the form of the requirement**. A dimension enters the acceptance criterion when it can be written as a condition on the *structure* of the artefact—an element that must be present, one that must be absent, a page of a given kind—and it stays outside when the only honest way to state it is a relation between elements that the interface has already flattened into a list. This is why the positive evidence concentrates in one component without the method being narrow. The misconception slot is not a lucky special case: it is the clearest instance of a requirement that names its own location in the artefact, which is the form that survives compilation. Coverage is the same property read where that form is available, and ordering where it is not; the two results are one finding about representation, seen from both sides. Narrowness, on this reading, is a fact about which curriculum dimensions have a structural form—not about how much of the course we happened to sample.

Mapping this onto the taxonomy of §1.2 assigns each kind a strength of evidence. *Cohort-specific misconceptions*: strongest. Given the text, `rag_mc` writes a correction slide in about half of nine generations and folds it into a bullet otherwise, after which it disappears; given the structure, `full` converts it into a layout-level requirement and is stable. Misconception knowledge must therefore not only be stated, it must be stated *into position*. *Skill landing points*: clear. Baselines omit skills such as implementing enqueue and dequeue on a list—not because the model cannot write code, but because it does not know this session requires *writing* rather than *understanding*; the 0.98 against 0.67 coverage gap comes largely from here, and on U12 it is 1.000 against 0.333. *Institution-specific order*: out of reach for this representation. Two reasons, and only the first is a fact about this course. The module's order matches the standard textbook, so a baseline that says nothing about order

is not disadvantaged; and the contract cannot withhold the order it states, because the concept list carries it (§1.5). A syllabus that *deliberately departs* from the textbook order would remove the first confound; it would not remove the second, which needs an interface that can present the concepts without sequencing them—a shuffled list, or a contract that names the successor relation rather than the total order. That is the follow-up we consider most worth doing, and it is a change to the representation, not to the sample.

6.2. What the graph changes: reliability, not average quality

The claim this paper defends is about reliability, and reliability is what this architecture could deliver. A system that produces a good deck two times in three is not a cheaper teacher; it is an unusable one, because the class arrives on schedule and a failure is not averaged away—it is absorbed by the person who has to prepare the session again. What separates the two conditions is not how much they say: `rag_mc` receives the same misconception sentence, in prose. It is that under the contract the sentence becomes a *requirement with a place*, so an omission is a defect to the same program that asked for the content, and there is no step at which the requirement can be renegotiated on its way to the check.

The two headline results are conventionally reported as means, but the informative difference is in the second moment. On misconception handling `rag_mc` averages 0.556 with a standard deviation of 0.527; `full` averages 1.000 with zero dispersion. What the graph changes is not *average quality* but the *reliability of quality*. In a teaching workflow a class happens on schedule and one failure means redoing the preparation, so a 0.556 ± 0.527 system is unusable and a 1.000 ± 0.000 system is usable; evaluation in this area should therefore report variance, not means alone, which the comparable literature we surveyed frequently does not. Table 5 also says which part of the contract buys that reliability: deleting the misconception slot alone returns the metric to 0.167, so the stability is a property of a component a reader can inspect and re-derive, not of the prompt as an undifferentiated whole.

The second course puts a condition on the sentence. There the slot moved the mean but not the dispersion (0.625 ± 0.479, §5.6): a contract can require a structural slot, but whether the slot is filled every time depends on the backbone's willingness to follow a structural instruction. §5.6 separates the two by re-running the first course, otherwise unchanged, under that backbone, and the same disappearance recurs with nothing but the endpoint changed. Reliability is therefore a property of the pair—representation and

model—and reporting it without naming the pair would repeat the mistake this section is about, which is reporting one moment of a distribution and calling it a result.

The same logic makes the negative results load-bearing. That we report no gain from ordering, no discrimination from editability, and a 33% non-pass rate for `full` itself is a credibility signal rather than a concession: the metrics do not systematically favour the constraint-based condition (seven defects were fixed to that end, §7.1), and the value of the knowledge base is concentrated in a narrow but critical gap that a better generator cannot close, because it is not a capability problem but a problem of information provenance. The ordering null is the clearest instance: it marks where the representation ends, which no table of wins could say.

7. Threats to validity

7.1. Text-level metrics systematically favour constraint-based methods

This is the most important methodological caution in the paper. During the study we discovered and corrected **seven** measurement defects (Table 9). Each made the `full` condition win without cause, and not one raised an error: the metrics kept emitting plausible numbers that happened to be wrong. None was found by reflection; all came from printing model outputs slide by slide and reading them.

The common structure across all seven is that **the representation the constrained method receives is exactly the representation the metric uses.** Any metric that scores against the list in the prompt encounters this problem.

Each row of Table 9 is now pinned by a regression test, one per defect, in `tests/test_measurement_defects.py`; the suite stands at 40 tests. Each test builds the deck that used to fool the metric and asserts the value it should produce—and, where the wrong arithmetic can be written down, asserts that arithmetic as well, so the test records what it is guarding against rather than only what it expects. A defect table without such guards documents a past bug instead of preventing its return, and all seven of these were silent when they were live.

One caution about reading the tables with defect 5 in mind. The 0.833 reported for `full`/U10/r3 in Table 4 is *not* a surviving instance of the defect: U10 is a six-concept session, and this is a genuine omission of the definition of "queue" (§5.8) that disappears when the title slide is admitted.

Table 9: Seven measurement defects that silently favour the constraint-based condition, and their fixes.

#	Defect	Symptom	Consequence	Fix
1	Verbatim coverage matching	Graph says "enqueue and dequeue operations", baseline writes "enqueue and dequeue"	Baseline coverage crushed to 0.50 while `full` scores 1.00 by copying source wording	Add aliases to the graph; one shared vocabulary across conditions
2	Order scored from self-reported IDs	Baselines never fill `concept_ids`	Empty sequence gives 0.00 violation; **an unanswered script scores full marks**	Score from textual position instead
3	Preview slides counted as body	Title and objectives slides list every concept	Ordering determined by list order rather than teaching order; three spurious violations in U10	Exclude title, objectives and summary
4	Forward reference treated as teaching	One slide remarks that modular arithmetic is key to implementing the circular queue	The concept is located three slides early; two spurious violations in U11	Locate by title naming first, fall back to first mention
5	Cross-slide subtraction	The simple term is cancelled by a compound term on another slide	The concept is scored as never taught although the deck does teach it	Compute net occurrences per line
6	Misconception anchors too broad	Unit-level `addresses_misconception` used as a slide anchor	Any slide with the same layout counts as corrected; the ablation still scores 1.00	Restrict anchors to concept level; fix one mis-attached data edge
7	Out-of-graph test too strict	Baselines invent their own ID scheme	Out-of-graph rate inflated to 0.67 when the only difference is a namespace	Count only non-existent IDs matching the graph's ID format

Defect 6 is not a bug in the metric code but **an error in the graph data**: an edge attached from a breadth-first-search misconception to a concept about empty-queue detection, on the reasoning that they were "related". They are not, instructionally; the consequence was that any misconception-layout slide mentioning empty-queue detection counted as addressing the other misconception, so the ablation still scored full marks and the conclusion about misconception injection nearly became false. **Graph errors silently contaminate metrics**, a risk specific to graph-driven methods. We therefore recommend that work of this kind report how the metrics behave *on the baselines* and not only the final numbers, and do at least one manual pass with outputs printed slide by slide.

7.2. Sample and external validity

Three sessions in one course, two in a second course in another language, three endpoints on the first course and one of them on the second, two to three resamplings per cell: this supports descriptive reporting and does not license significance claims. The second course contributes four runs per condition, which is enough to see that a distribution became two-of-four and not enough to estimate how often it does so—a sign, not a rate. Seven gaps follow, each with a different remedy.

One factor at a time. Each pair varies one thing while holding another fixed (`full` vs `rag_mc` holds text and varies structure; `full` vs `placebo` holds length), but the factors are never crossed, so main effects are estimable and interactions are not. It remains formally possible that the misconception gain requires both structure and text and would vanish if either were removed. We read the pattern as additive because both controls are individually null, but that is an inference from two marginal comparisons, not a factorial result.

Two models, not a model family. All runs come from hosted endpoints at one point in time. §5.5 re-runs the claim-bearing conditions on a second vendor's model and the contrast survives, but two draws do not characterise a space. Ordering was already handled well by both models; we have no evidence about weaker open-weight models, where the graph might help on ordering too, or stronger ones, where the misconception gap already narrowed from 0.444 to 0.333 and could close. The study is reproducible in procedure and not in output: a hosted endpoint may retire a model without notice, and Kimi K3 already refuses the temperature the main table uses.

Course and backbone, now separated but not crossed. The second course changes syllabus, language and endpoint together, which on its own would

leave the failure of variance replication unattributable to either. §5.6 closes that hole from one side: re-running the first course unchanged under the second course's backbone reproduces the failure, so the condition sits with the model. What remains open is the fourth cell of the 2×2—the second course under DeepSeek—which we could not run because that endpoint's balance was exhausted on the day, and the interaction it would measure: whether the slot is reliable on a course whose misconceptions are measured rather than documented, under a backbone that has so far filled it in every run.

Two courses, two modules. Both modules are unusually misconception-rich (empty/full, false overflow and BFS semantics in the first; 1-based indexing and the null terminator in the second), close to a best case. A course with a thin misconception inventory should be expected to show a smaller effect, and we claim nothing about its size.

One graph, one author. The graph was authored by the instructor who teaches the course; its edges were not independently annotated, so we cannot report inter-annotator agreement on the graph itself. Defect 6 is the concrete consequence—one mis-attached edge nearly inverted a headline result.

No learning outcome. Nothing here measures whether students learn more from a constrained deck. Every metric, including the teacher review, scores the artefact; the chain from artefact quality to learning gain is untested, and it is the chain a teacher cares about.

Under-powered ablations, one of them null. Six runs per arm separate a metric that moves from 0.000 to 0.083 and cannot resolve one that moves by 0.032; and the ordering arm removes less than its name claims, since the concept list it leaves behind is already topological, so the order survives its own deletion (§5.4). An ablation of something the prompt states twice has to remove both statements before it tests anything. We treat this as a limit of the interface disclosed in §1.5 rather than as a gap that more runs would close: more runs would estimate the price of a duplicate more precisely, not the price of the constraint.

7.3. What the human review can and cannot support

The blind review of §5.7 narrows this gap without closing it. Ten decks, three raters, one institution and a single `placebo` deck support a consistency check, not a validation study; the human–automatic agreement is measured at condition level on four points, enough to say the two orderings coincide and not enough to estimate a correlation. Nor do the items exhaust teaching

quality: clarity of board work, appropriateness of examples for this cohort, and the fit between slides and the teacher's own voice all remain outside both the metrics and the rubric. What the review does establish is that teachers order the conditions as the metrics do and locate the difference on the same axis—misconception handling and coverage—while rating every condition as factually accurate. A larger review with raters from other institutions is the obvious next step and the place where our evidence is thinnest.

7.4. A repair loop, and where it stops

The validator emits findings, and in the deployed workflow a teacher acts on them; in the main experiment nothing did. One third of the `full` runs contain at least one error-level finding (§5.4), so a reader is entitled to ask how many of those a second pass would fix. We ran that loop on the stored runs of two cells: the DeepSeek grid of Table 4 (nine runs, three failures) and the `agnes-2.5-flash` cell of Table 7 (row 3; six runs, two failures)—the cell where the misconception slot's advantage disappears, and therefore the harder place to ask.

The policy is fixed and deliberately plain. For a run with an error-level finding, the validator's error findings are appended to the same condition prompt and the whole deck is regenerated, up to three attempts; a run that passed first time is not touched, so the repaired count shares its denominator with the first-pass count. The appended text names the uncovered concepts and the inverted pairs and adds nothing else—for `full` those names are already in the contract, so a finding restates a requirement instead of supplying one. The same loop on `zero_shot` would hand over the syllabus that condition exists to lack, which is why it is not run there: whether a repair loop is fair is a property of the condition, not of the loop.

On the main grid the loop closes the gap. All three failures were repaired in a single round each: $6/9 \rightarrow 9/9$, mean coverage $0.981 \rightarrow 1.000$, three model calls in total. The repairs are substantive rather than metric artefacts, and the two failure types were repaired differently. A missing concept was fixed by giving it the page it lacked: in U10/r2 the prerequisite `C1002` (queue front and rear) existed only inside a later summary bullet, and the second attempt made it a page of its own. An inverted pair was fixed by splitting a page that had merged two concepts—U11/r3 had put `C1105` (queue length) and `C1106` (the implementation) on one code page whose title named the second one first—so the metrics are computed on more content

than before, not less: page count is unchanged and coverage is 1.000 after repair in all three.

On the third backbone it does not. The same policy on the `agnes-2.5-flash` cell moved 4/6 to 5/6. The missing-concept failure was repaired in one round; the ordering failure was not repaired in three, and its violation rate went $0.048 \to 0.048 \to 0.381$, worse on the third attempt than on the first. One call fixed one of the two; three more were spent on the run that did not converge.

The split is the paper's own precondition, seen for the third time. **The loop closes every failure on the backbone that fills a required slot and one of two on the backbone that does not**; the same division appears for the slot itself in §5.6. A repair loop is therefore not a separate capability that rescues a weak condition—it inherits the contract's dependence on the backbone, and a reader deciding whether to build one should first ask whether their model executes structural instructions at all.

Two cautions. Three successes out of three is a count, not a rate: its exact 95% lower bound is $0.05^{1/3} \approx 0.37$, so the loop is shown to work and not shown to be reliable, and a larger grid could easily find a deck it cannot fix. And three attempts on one deck produced three different decks with no convergence, so a repair budget cannot rest on the expectation that it will converge; it has to be set by cost. Table 4 therefore keeps 67% as a first-pass figure, and 9/9 is reported as the outcome of a bounded, costed repair on nine decks—not as the pass rate of the method.

7.5. Evaluation independence

The same author wrote the graph, the method, the metrics and the rubric—the standard shape of a systems paper and its standard weakness. Three countermeasures were taken and none substitutes for an independent coder. Every metric is computed by released code against a released contract, and the seven defects were found by printing model output and reading it rather than by inspecting the code, which is adversarial but self-administered. The rubric was applied identically to all conditions with one shared vocabulary, and baselines were never told which fields the metrics read; the `placebo` arm exists to test whether the metric measures prompt length and fails to explain the gain. The blind review puts the ranking question to three raters who did not know the conditions, and they reproduce it—evidence about the decks, not about the rubric's provenance.

The rubric's sensitivity is reported rather than defended: two of its three decisions were re-run under the alternative convention (§5.3), the conclusion survives both, and the reported advantage sits on the conservative side. What is missing is the obvious thing—a second coder who did not write the metric, coding a subsample against the same rubric with disagreement reported. Given three decisions that materially move the numbers, that check would be worth more than a larger sample scored by the same author.

7.6. Code is parsed, and only partly executed

For a programming course this is the weakest joint. Every one of the 54 runs carries code, with `full` putting it on 48% of its pages against 27–33% for the baselines, yet no headline metric parses, lints or executes it: a slide can score coverage 1.000 while its `enqueue` has an off-by-one, and nothing in Table 4 would move.

The cheapest check costs one line and we ran it: all 117 blocks parse with Python's `ast`, in every condition. That is a null result and a useful one—it is consistent with teachers rating content accuracy at or above 4.0 everywhere (Table 8), and it means a runnability metric must test behaviour to discriminate at all.

Behaviour is testable within limits, and we tested it (`paper/behavior_check.py`; full protocol and per-condition results in Table S3 of the supplementary material). Each block is stripped of its top-level demo statements, executed in a fresh namespace and interrogated on up to four behaviours: FIFO order, the empty-dequeue signal, the full signal, and survival of repeated enqueue. Thirty-seven of 117 blocks (0.32) qualify; 36 satisfy every behaviour asked of them, and the one failure is a genuine scaffolding defect rather than a probe artefact—a program-style `enqueue(q, x)` referencing a capacity constant its own slide never defines. This is a bound, not a metric: the harness is ours, so the evaluator supplies part of the answer it then checks, and the qualifying denominator is too small and too condition-dependent to rank arms—`full` passing 8 of 9 against 28 of 28 for the baselines is one block in nine and separates nothing. What it establishes is that the code dimension is not silently broken; a comparison would need a per-session harness and test set, released. Until then the honest description is *one syntactic check, one bounded behavioural probe, and manual inspection in the qualitative pass* (§5.8).

7.7. The constructedness of the misconception result

`misconception_coverage` is structurally unfavourable to baselines, since they were never told about the misconceptions. Our response is the `rag_mc` control, which supplies the same text in flat form. If `full` still wins, the gain comes from structure rather than information. Even so, we must concede that the metric measures *whether* the course-required correction was made, not *whether it is correct*.

8. Conclusion and future work

We reframe slide generation as a **course-specification satisfaction problem** and make the criterion, not the generator, the contribution: one graph-derived contract serves as generation basis and acceptance ground truth, so what is demanded and what is checked are the same object and cannot drift, and alignment is measured per artefact instead of asserted per course design.

Empirically the gain is a gain in *reliability*—a misconception slot that is always filled rather than usually filled—and the knowledge the graph makes irreplaceable is *the skill landing points in the syllabus* and *the cohort-specific misconceptions*, neither of which exists in any pretraining corpus. Each gain is priced against a named block rather than against the pipeline as a whole (Table 5), which is what lets a reader inspect the claim instead of taking it on trust. The same study locates the edge of that claim: order is expressible in the graph but cannot be withheld by the interface, so it is reported as the boundary of the representation rather than as a result about ordering.

Future work proceeds along five lines: (i) an interface that can present concepts without sequencing them, so that ordering becomes testable at all; (ii) extend to all sessions of the course and widen the blind review to raters from other institutions, courses and vocational disciplines; (iii) build graphs semi-automatically from textbook PDFs to reduce authoring cost; (iv) add a render-and-review loop (screenshot, detect overflow and overlap, repair automatically); and (v) release a Chinese vocational slide-generation benchmark so that comparable work can be compared on common ground.

CRediT authorship contribution statement

Suxing Liu: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review and editing, Visualization.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

Code, both curriculum graphs, prompts, experimental protocol and all raw model outputs are archived at `https://doi.org/10.5281/zenodo.XXXXXXX`. The same material, together with its development history and the scripts that regenerate every table and figure in this paper, is public at `https://github.com/HAHA1122344/kg-slidegen`; the archive is a packaged snapshot of that repository for citation, not a separate copy. Each experimental cell retains three artefacts: the rendered deck, the raw model output, and an itemised validation report. Every table in this paper and in the supplementary material is regenerated from that archive by the scripts in `paper/`, and `tests/` reproduces the graph's own validation results (Table S2). The archive is byte-reproducible—packaging the same tree twice yields the same bytes—and the record's file manifest carries the archive's `sha256`, so a reader can confirm they hold the reviewed artefact rather than a later rebuild.

Supplementary material

The supplementary material (Sections S1–S5, Tables S1–S6) holds the ablation results, the mutation test of the graph's structural checks, the full behavioural probe of the generated code, the cross-model replication, and the concept-level paired analysis with its leave-one-concept-out check, together with the scripts that regenerate each of them from the archive.

Appendix A. The prompt for one session

The course is taught in Chinese, so the prompt is written in Chinese; that is a property of the deployment, not a choice we made for the experiment. Verbatim prompts for the `full` and `rag_mc` conditions of session U11 are archived as `paper/prompt-U11-full.txt`; the rendering below keeps every section header, every constraint and every hard rule, and condenses only the per-item lists that the archive reproduces in full.

> **Instruction.** Generate teaching slides according to the following constraints.
>
> **Session.** U11 Circular queue (7 pages).
>
> **Teaching objective.** Understand how a circular queue reuses space through modular arithmetic, distinguish the empty and full conditions correctly, and write the implementation independently.
>
> **Concepts that must be covered (none may be omitted).** Seven entries, each with identifier, name, importance level, optional complexity and a one-sentence definition, for example: `C1104 Full condition (proficient)`: to separate "full" from "empty" one storage cell is usually sacrificed, so the queue is full when `(rear + 1) % maxsize == front`.
>
> **Explanation order (mandatory; this is the topological order induced by the prerequisite relation).** The seven identifiers in sequence, with arrows.
>
> **Misconceptions that must be corrected.** One entry: the erroneous claim (the queue is full when `rear == front`), followed by the correction and the consequence of confusing the two.
>
> **Accompanying lab.** Implement a circular queue of capacity 8, run 20 enqueues and 15 dequeues, and verify modular reuse and the full condition.
>
> **Evidence anchors (for teacher verification, not generated content).** Six entries of the form `C1101: handout §4.3: the circular queue exists precisely to solve false overflow`.
>
> **Previously taught concepts that may be recapped in one sentence but not expanded.** Ten sibling concept names, followed by the constraint that they must not exceed 25% of the text.

Concepts that must not be introduced. Five names (priority queue, monotone queue, message queue, shortest path, topological sort).

Output format. A single JSON object with `unit_id` and a `slides` array; each slide carries `index`, `layout` (one of seven), `title`, `bullets`, `code`, `notes`, `concept_ids` and `background`. No Markdown fence, no commentary.

Hard requirements. (1) Seven pages, first a title page and last a summary. (2) Every concept in the mandatory list must appear in some page's `concept_ids`. (3) Page order must follow the given explanation order; no prerequisite may be inverted. (4) At most four bullets per page, at most forty characters each. (5) If misconceptions are listed, at least one page must have layout `misconception`. (6) Do not introduce forbidden concepts; if one must be mentioned, set `background` to true.

The `rag_mc` control replaces everything from *Session* to *Forbidden concepts* with a flat list of the same definitions under the heading *Textbook excerpt*, and keeps the output format and hard rules identical. That is the point of the control: the information is held constant and only its structure changes.

Appendix B. One raw response, and what the validator saw

Responses are Chinese for the same reason the prompts are. The first two slides of one `full` response (U11, run 1) are: index 1, layout `title`, "Circular queue", three bullets naming the lecture, the objective and the lab, no code, `concept_ids [C1101]`; index 2, layout `concept`, "The circular queue and modular arithmetic", four bullets on vacated cells, the logical ring, `(index + 1) % maxsize` and the $O(1)$ bound, code carrying the two pointer-advance lines. The validator records coverage 1.000, prerequisite violation 0.000, misconception coverage 1.000, out-of-graph 0.000, background 0.012 against a 0.25 ceiling, and editability 1.000, with no findings. The report is addressed to concepts, not to a score: a failure names the uncovered concept ids, the out-of-graph terms and the inverted pairs, so a low number can be traced to a slide rather than only observed. The verbatim JSON and report are in the archive under `outputs/main_v1/`.

Appendix C. Reproducibility

All runs were issued against the vendor's hosted API (`https://api.deepseek.com`) with model id `deepseek-v4-flash`, temperature 0.3, the provider's default thinking mode, and no fixed seed; the model therefore returns different slide text on repeated calls, which is why each cell is resampled three times rather than run once. Slides are emitted as JSON and rendered with `python-pptx`; no image generation model is involved, which is what makes `editability` 1.000 in every condition.

Code, graph, experimental protocol and all raw outputs are available in the `kg-slidegen` repository. The accompanying protocol document records how every number in this paper was produced, and each experimental cell retains three artefacts: the presentation file, the raw model output and a per-item validation report. The teacher ratings are released as `paper/human_eval.csv`, one row per deck–rater pair, and `paper/human_eval.py` regenerates Table 8, the ICC values and their bootstrap intervals from it.

References

[1] H. Liang, Q. Lin, Z. Han, X. Ma, Z. H. Wong, M. Qiang, L. Sun, W. Zhang, K12-KGraph: A curriculum-aligned knowledge graph for benchmarking and training educational LLMs, in: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2026, arXiv:2605.09635, v4 of 3 Oct 2026; the graph and K12-Train grew between v1 and v4. doi:10.48550/arXiv.2605.09635.

[2] X. Dai, Y. Hua, T. Wu, Y. Sheng, Q. Ji, G. Qi, Large language models can better understand knowledge graphs than we thought, Knowledge-Based Systems 312 (2025) 113060. doi:10.1016/j.knosys.2025.113060.

[3] W. Huang, G. Zhou, M. Lapata, P. Vougiouklis, S. Montella, J. Z. Pan, Prompting large language models with knowledge graphs for question answering involving long-tail facts, Knowledge-Based Systems 324 (2025) 113648. doi:10.1016/j.knosys.2025.113648.

[4] J. Ma, Z. Zhao, Z. Xie, Y. Zhang, G. Zhou, LE-DLCM: Decoupled learner and course modeling with large language models for enhanced course recommendation, Knowledge-Based Systems 334 (2026) 115135. doi:10.1016/j.knosys.2025.115135.

[5] J. Roschelle, E. A. McLaughlin, K. R. Koedinger, Beyond benchmarks: Responsible AI in education needs learning sciences, Communications of the ACM 68 (12) (2025) 37–39. doi:10.1145/3747174.

[6] T.-J. Fu, W. Y. Wang, D. McDuff, Y. Song, DOC2PPT: Automatic presentation slides generation from scientific documents, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022. doi:10.1609/aaai.v36i1.19943.

[7] I. Mondal, S. Shwetha, A. Natarajan, A. Garimella, S. Bandyopadhyay, J. Boyd-Graber, Presentations by the humans and for the humans: Harnessing LLMs for generating persona-aware slides from documents, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), St. Julian's, Malta, 2024, pp. 2664–2684. doi:10.18653/v1/2024.eacl-long.163.

[8] J. Ge, Z. Z. Wang, X. Zhou, Y.-H. Peng, S. Subramanian, Q. Tan, M. Sap, A. Suhr, D. Fried, G. Neubig, T. Darrell, AutoPresent: Designing structured visuals from scratch, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025, pp. 2902–2911. doi:10.1109/CVPR52734.2025.00276.

[9] H. Zheng, X. Guan, H. Kong, W. Zhang, J. Zheng, W. Zhou, H. Lin, Y. Lu, X. Han, L. Sun, PPTAgent: Generating and evaluating presentations beyond text-to-slides, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Suzhou, China, 2025, pp. 14402–14418. doi:10.18653/v1/2025.emnlp-main.728.

[10] Y. Guo, Z. Zhang, Y. Liang, D. Zhao, N. Duan, PPTC benchmark: Evaluating large language models for PowerPoint task completion, in: Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 2024, pp. 8682–8701. doi:10.18653/v1/2024.findings-acl.514.

[11] X.-S. Chen, J. Zhu, P.-l. Li, H. Wang, S. Yang, M. Guo, PresentBench: A fine-grained rubric-based benchmark for slide generation, 238 instances, 54.1 checklist items per instance (2026). arXiv:2603.07244, doi:10.48550/arXiv.2603.07244.

[12] D. Jang, M. L. Heisler, L. Xing, Y. Li, E. Wang, Y. Xiong, Y. Zhang, Z. Fan, DeckBench: Benchmarking multi-agent frameworks for academic slide generation and editing (2026). arXiv:2602.13318, doi:10.48550/arXiv.2602.13318.

[13] Z. Huang, X. Liu, T. Hu, K. Zhang, Y. Liu, PPTBench: Towards holistic evaluation of large language models for PowerPoint layout and design understanding (2025). arXiv:2512.02624, doi:10.48550/arXiv.2512.02624.

[14] K. Kumar, C. R. Chowdary, SlideSpawn: An automatic slides generation system for research publications (2024). arXiv:2411.17719, doi:10.48550/arXiv.2411.17719.

[15] K. Jung, H. Cho, J. Yun, S. Yang, J. Jang, J. Choo, Talk to your slides: High-efficiency slide editing via language-driven structured data manipulation, in: Findings of the Association for Computational Linguistics: ACL 2026, San Diego, California, 2026, pp. 3370–3399. doi:10.18653/v1/2026.findings-acl.166.

[16] T. Lu, Z. Wang, Y. Zhou, Y. Guan, Z. Bai, J. Du, SciMKG: A multimodal knowledge graph for science education with text, image, video and audio, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 15466–15474. doi:10.1609/aaai.v40i18.38574.

[17] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph RAG approach to query-focused summarization (2024). arXiv:2404.16130, doi:10.48550/arXiv.2404.16130.

[18] Y. Huang, J. X. Huang, A survey on retrieval-augmented text generation for large language models, ACM Computing Surveys 58 (12) (2026) 1–38. doi:10.1145/3805774.

[19] A. Asai, Z. Wu, Y. Wang, A. Sil, H. Hajishirzi, Self-RAG: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations, 2024. doi:10.48550/arXiv.2310.11511.

[20] S. Santschi, Think tasks, not tools: why AI policy in higher education needs constructive alignment, Teaching in Higher Education (2026) 1–11doi:10.1080/13562517.2026.2671391.

[21] B. Divjak, B. Svetec, D. Horvat, N. Kadoić, Assessment validity and learning analytics as prerequisites for ensuring student-centred learning design, British Journal of Educational Technology 54 (1) (2022) 313–334. doi:10.1111/bjet.13290.

[22] C. Wekerle, M. Sailer, I. Kollar, T. Fütterer, The effects of ICAP learning activities on learning outcomes, Learning and Instruction 107 (2026) 102482. doi:10.1016/j.learninstruc.2026.102482.

[23] A. Kozanitis, L. Nenciovici, Effect of active learning versus traditional lecturing on the learning achievement of college students in humanities and social sciences: a meta-analysis, Higher Education 86 (6) (2022) 1377–1394. doi:10.1007/s10734-022-00977-8.

[24] C. Dong, Y. Li, H. Gong, M. Chen, J. Li, Y. Shen, M. Yang, A survey of natural language generation, ACM Computing Surveys 55 (8) (2022) 1–38. doi:10.1145/3554727.

[25] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, ACM Computing Surveys 55 (12) (2023) 1–38. doi:10.1145/3571730.

[26] W. Yu, C. Zhu, Z. Li, Z. Hu, Q. Wang, H. Ji, M. Jiang, A survey of knowledge-enhanced text generation, ACM Computing Surveys 54 (11s) (2022) 1–38. doi:10.1145/3512467.

[27] S. Geng, M. Josifoski, M. Peyrard, R. West, Grammar-constrained decoding for structured NLP tasks without finetuning, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 2023, pp. 10932–10952. doi:10.18653/v1/2023.emnlp-main.674.

[28] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, RAGAS: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th

Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, St. Julians, Malta, 2024, pp. 150–158. doi:10.18653/v1/2024.eacl-demo.16.

[29] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, I. Stoica, Judging LLM-as-a-judge with MT-Bench and Chatbot Arena, in: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, Vol. 36, 2023. doi:10.52202/075280-2020.

[30] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu, G-Eval: NLG evaluation using GPT-4 with better human alignment, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 2023, pp. 2511–2530. doi:10.18653/v1/2023.emnlp-main.153.

[31] Learning Commons, Knowledge graph and evaluators: open infrastructure for grounding AI educational tools in curricula, https://chanzuckerberg.com/education/ai, chan Zuckerberg Initiative; accessed 2 November 2026 (2026).

[32] J. Sweller, An integrated human cognitive architecture, Educational Psychology Review 37 (4) (2025). doi:10.1007/s10648-025-10089-1.

[33] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837. doi:10.52202/068431-1800.

[34] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations, 2023. doi:10.48550/arXiv.2210.03629.